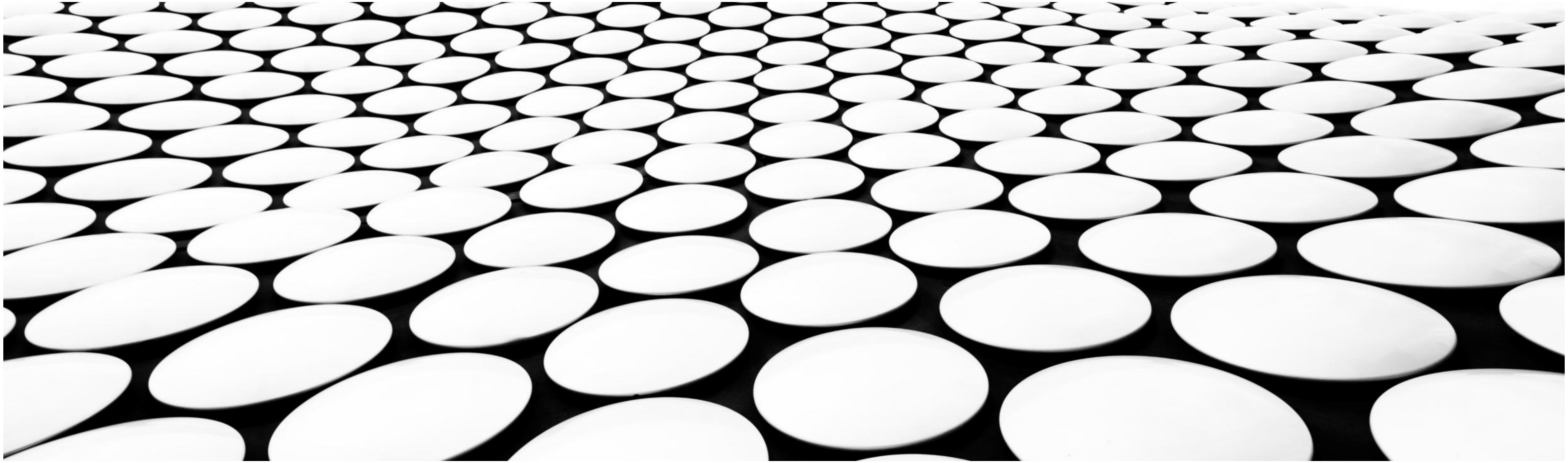
RECOMMENDER SYSTEMS

An overview of different approaches

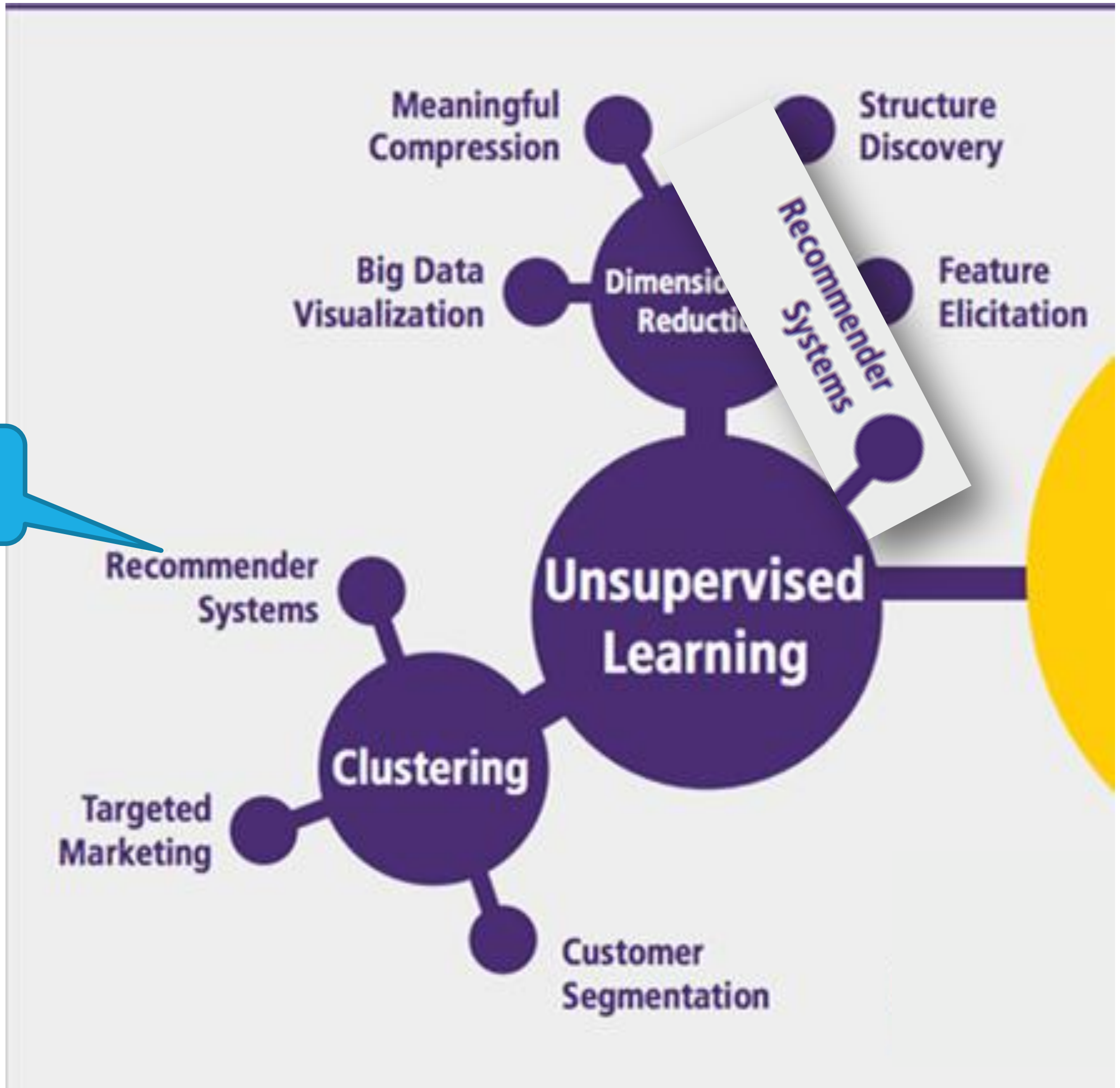


RECOMMENDER SYSTEMS

- Introduction
- Content-Based Filtering
- Collaborative Filtering
  - Utility matrix
  - Memory-based
  - Model-based
- Evaluation

INTRODUCTION

We focus here



References

Very slow… but explains the matrix factorization idea

How does Netflix recommend movies? Matrix Factorization

Serrano.Academy
168 k abonnés
Rejoindre
S'abonner
9,8 k

Source

Recommendation Systems: An Overview

Sharan Harsoor · Follow
Published in AI Mind · 25 min read · Nov 13, 2023

Source

DATA SCIENCE

Recommender Systems – A Complete Guide to Machine Learning Models

Leveraging data to help users discovering new contents

Francesco Casalegno
Nov 25, 2022 • 11 min read

Source

thingssolver

Introduction to recommender systems

13. 01. 2021.

Source

Definition

Recommender systems are algorithms providing personalized suggestions for items that are most relevant to each user.

With the massive growth of available online contents, users have been inundated with choices.

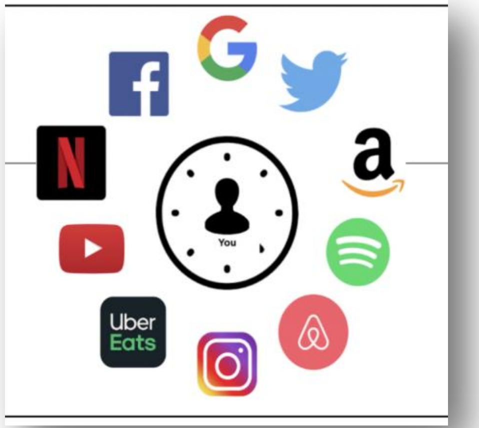
It is therefore crucial for web platforms to offer recommendations of items to each user, in order to increase user satisfaction and engagement.

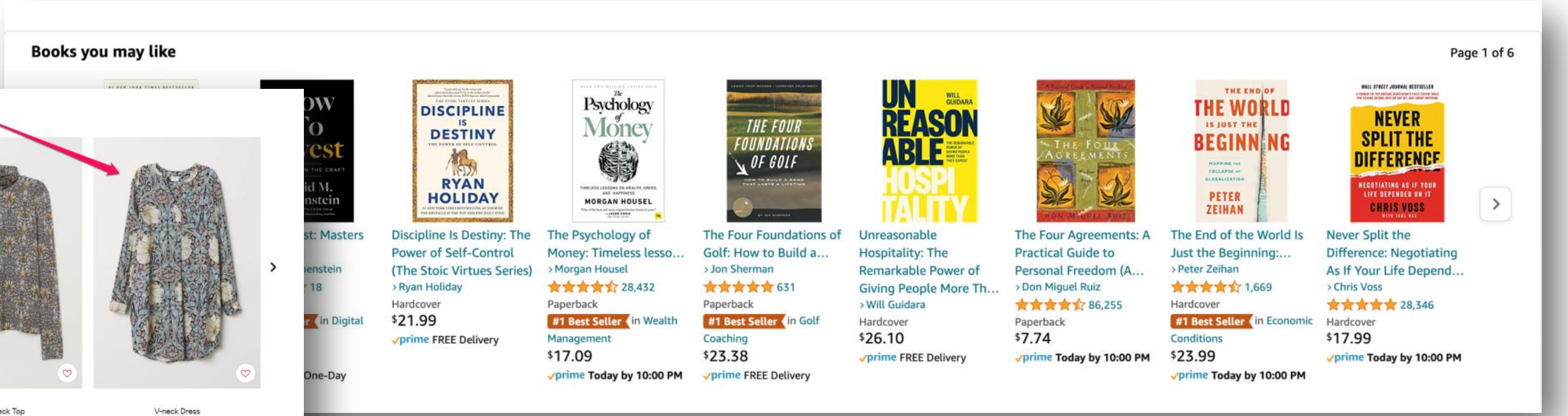
Source

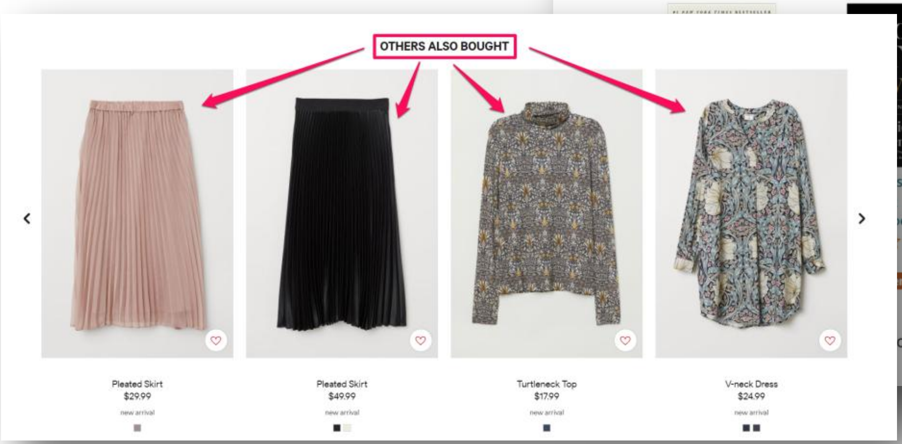
Applications

Recommendations can be used in so many domains:

- Restaurants
- Hotels
- Clothing items
- Food items
- Movies
- Music
- Posts
- ...





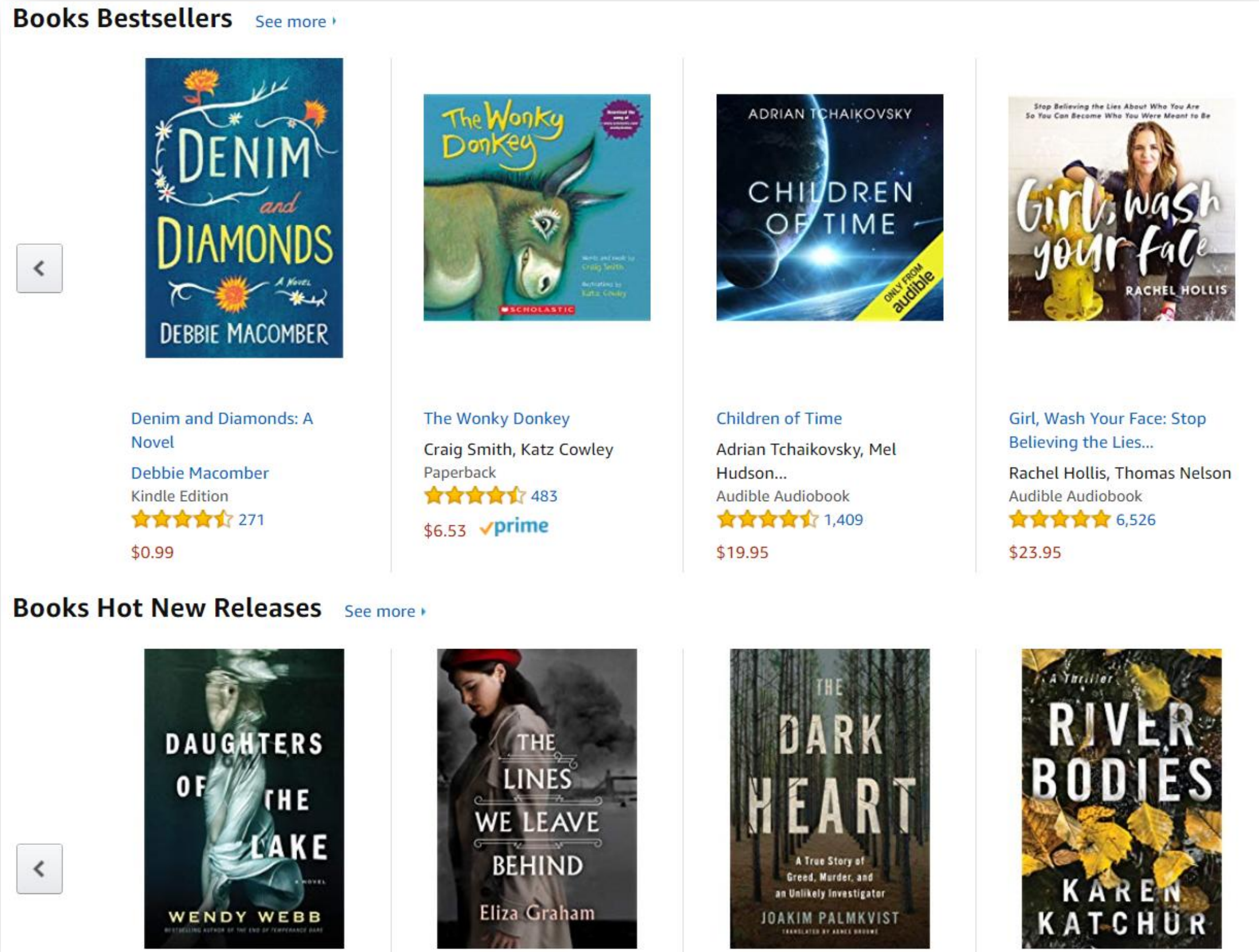


Types

Four types of recommendation systems, each relying on a type of filtering:

- Non-personalized filtering
- Search-based filtering
- Content-based filtering
- Collaborative filtering

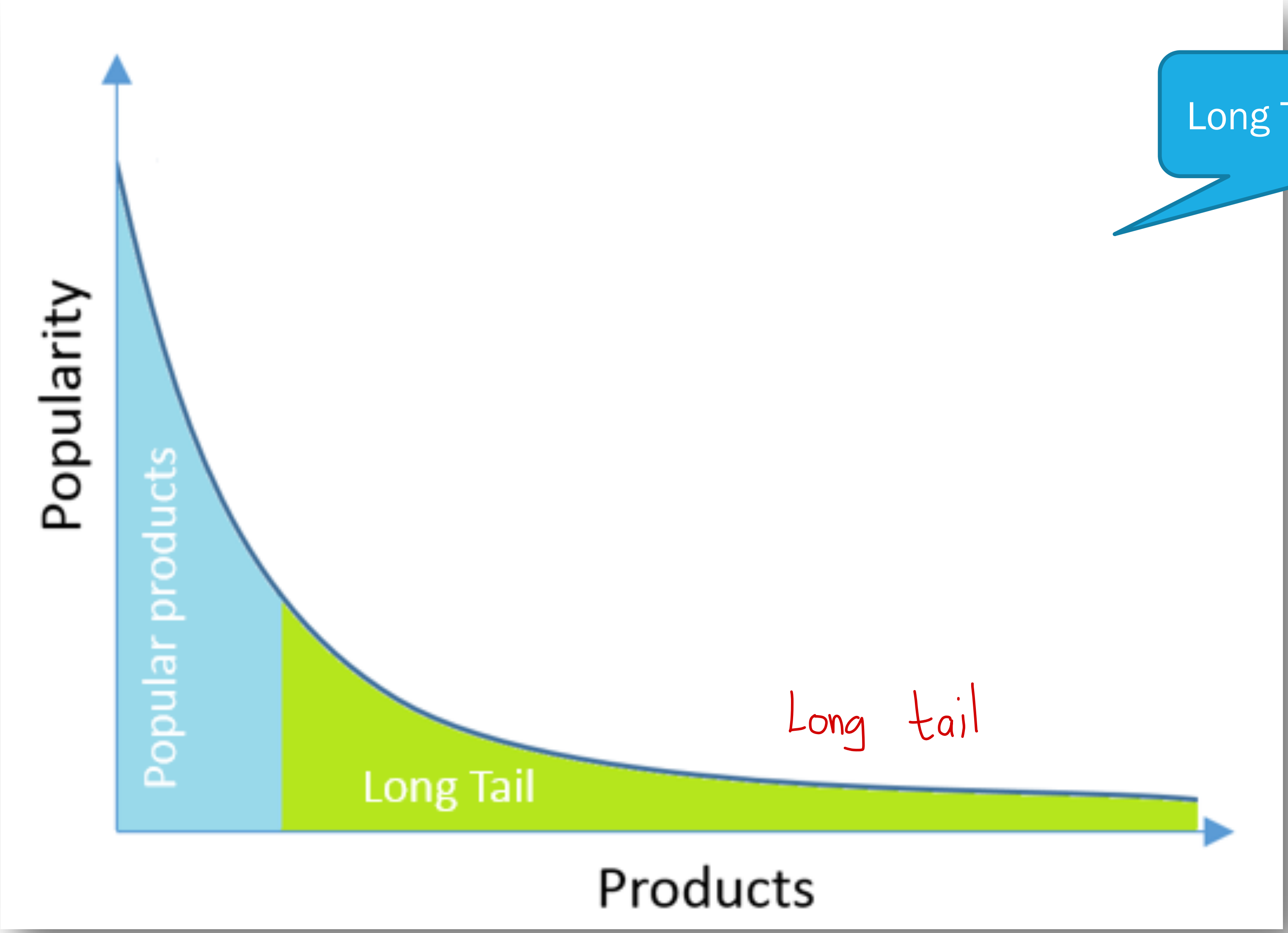
Non-personalized



Criteria

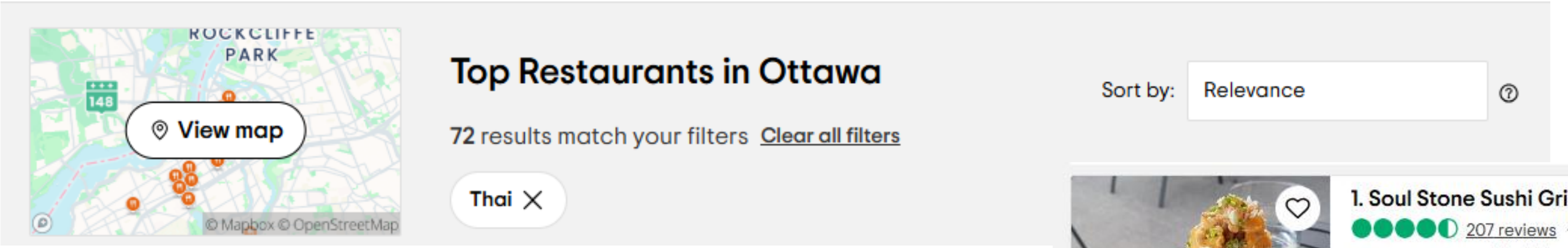
- Recency
- Popularity (days)
- Trending (hours)

Not related to you, related to other factors

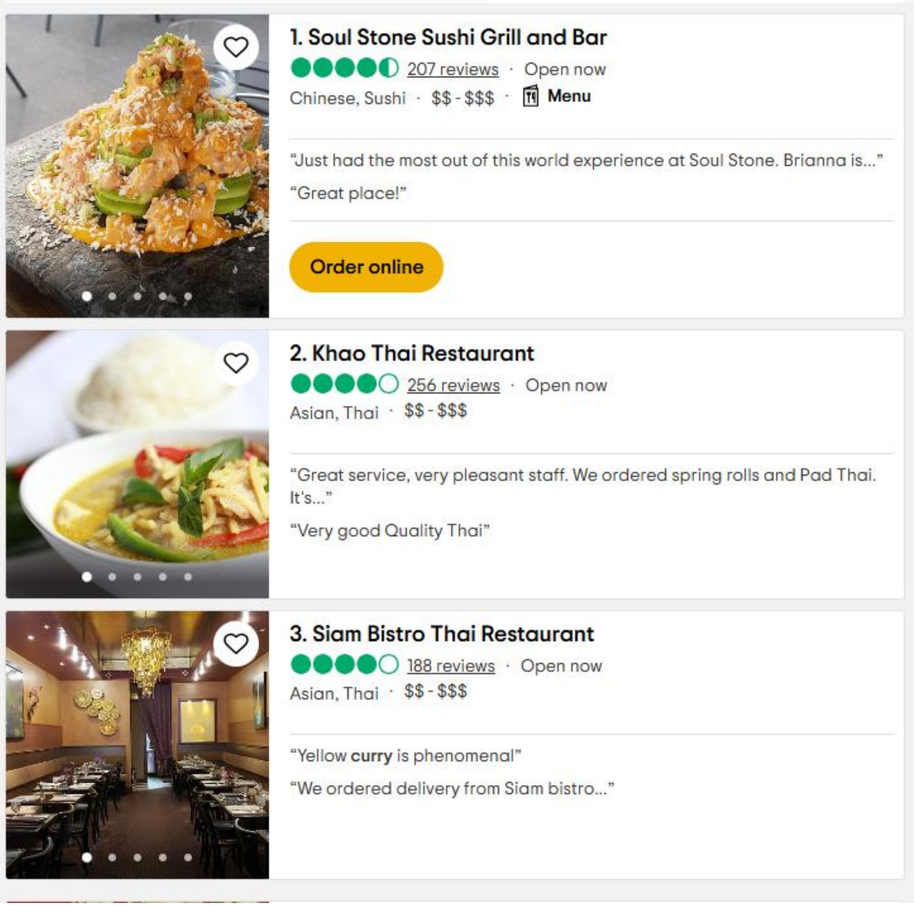


Long Tail Distribution

Tripadvisor



Search-Based Recommendation



Loblaws

Faceted Search

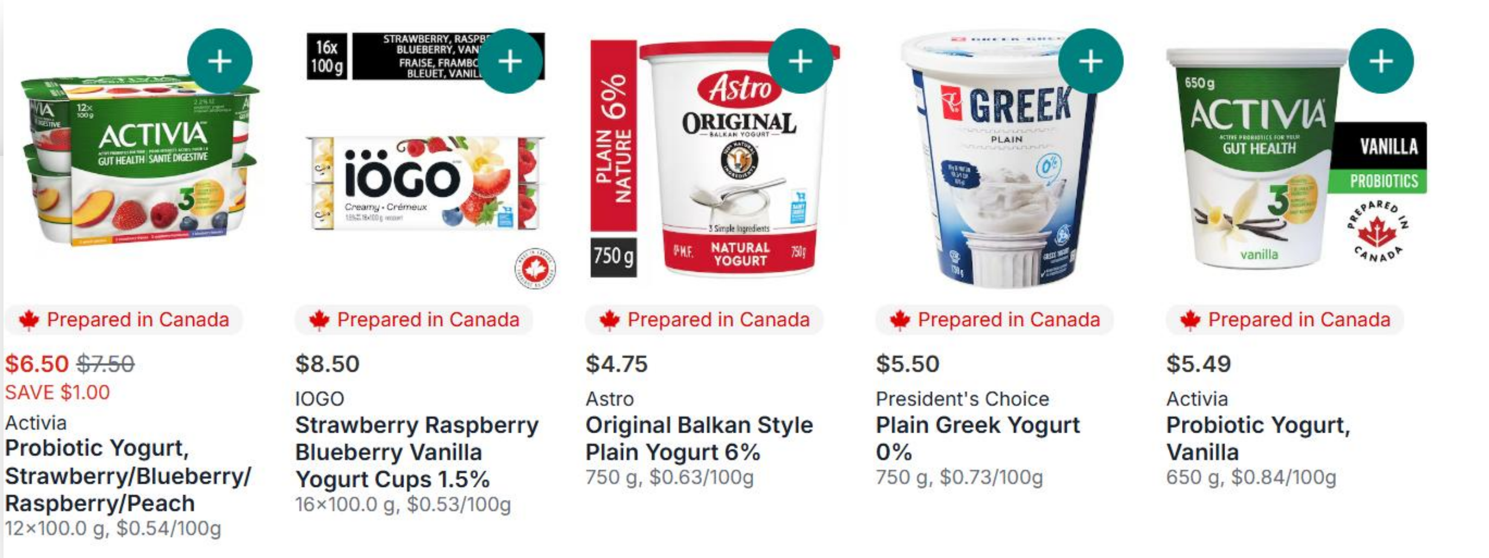
Brand

Search

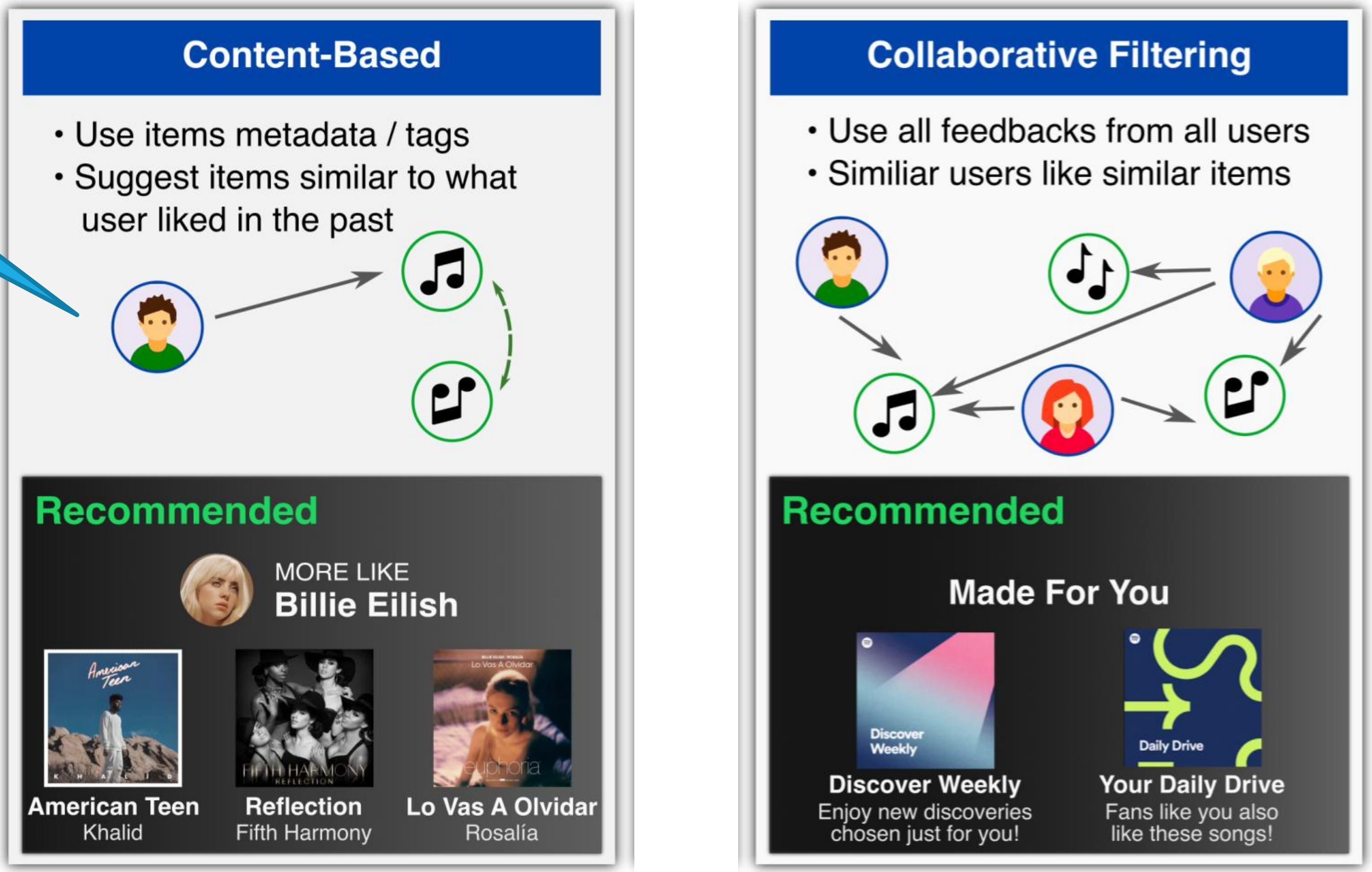
- Activia
- Astro
- Baby Gourmet
- Bio K+
- Chapmans
- Danone
- Dare
- Earth's Best
- Gerber

Dietary & Lifestyle

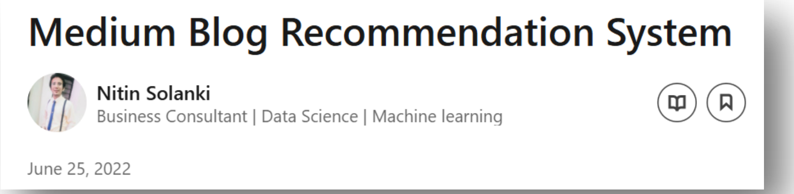
- Whole Grain
- Sugar Conscious
- Lactose Free
- Peanut Free Facility
- Low & Less Fat
- Vegan
- Source of Vitamins or Minerals
- Gluten Free
- Organic



Personalized recommendation



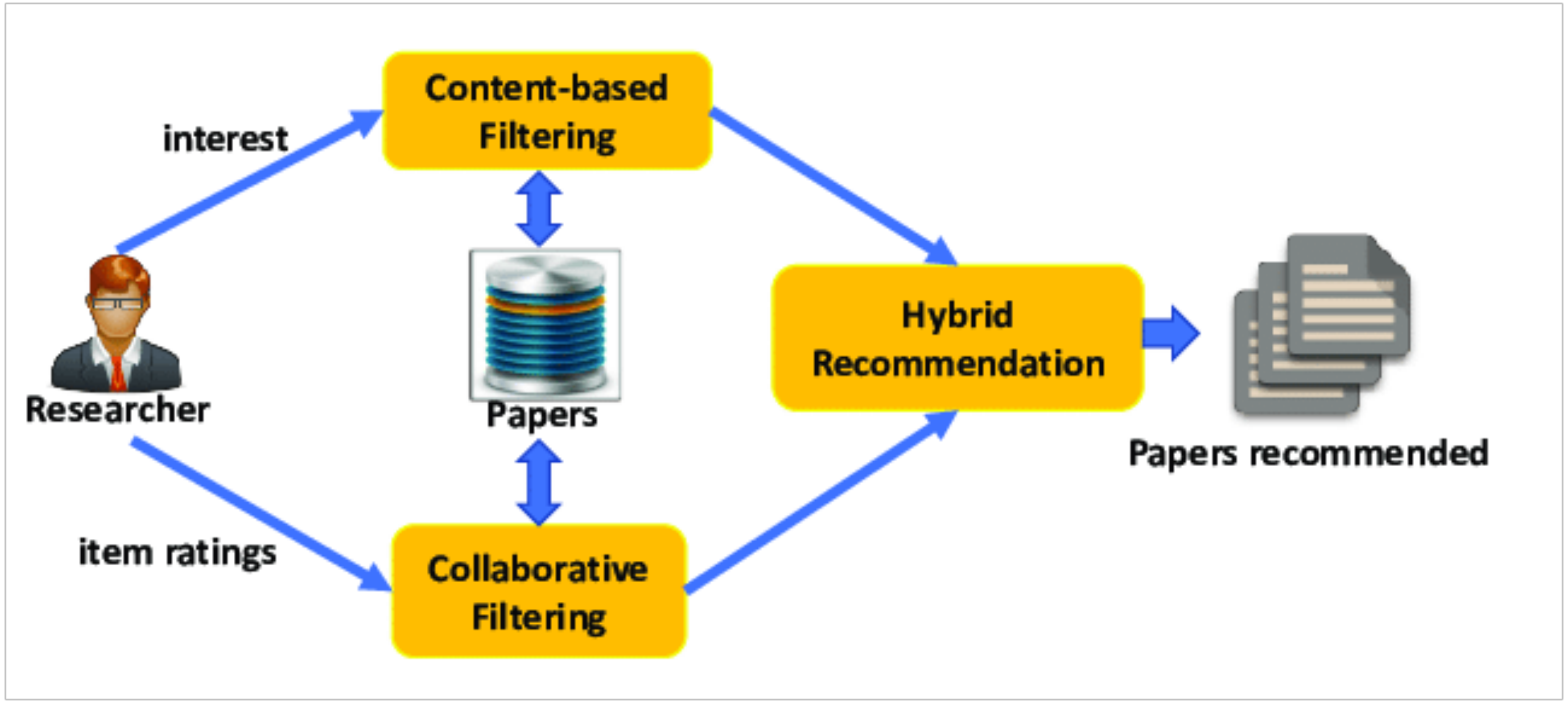
Hybrid



Source

Hybrid Methods:

Hybrid recommendation systems combine aspects of both collaborative and content-based filtering to provide more accurate and diverse recommendations. These systems aim to overcome the limitations of individual methods and improve overall performance.



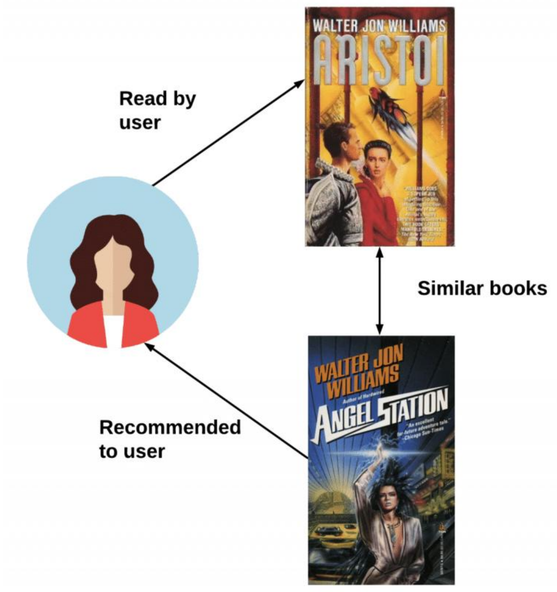
Content-Based Recommendation

<u>Content-based methods</u> describe **items by their known metadata**.

kindle direct publishing

[Source]

Metadata Guidelines for Books



Contents

- Book title
- Subtitle
- Series information
- Description
- Author
- Contributors
- Publisher
- Keywords
- Primary audience
- Primary marketplace
- Categories
- Cover image
- ISBN

Medium Blog Recommendation System

Nitin Solanki
Business Consultant | Data Science | Machine learning

June 25, 2022

Source

Average Reading time

Data 5 min read · Selected for you

Clap and comment

1.7K 35

Blog Title

Khushee Kapoor · Jan 7

Market Basket Analysis: Association Rule Mining

Tags

Add or change tags (up to 5) so readers know what your story is about

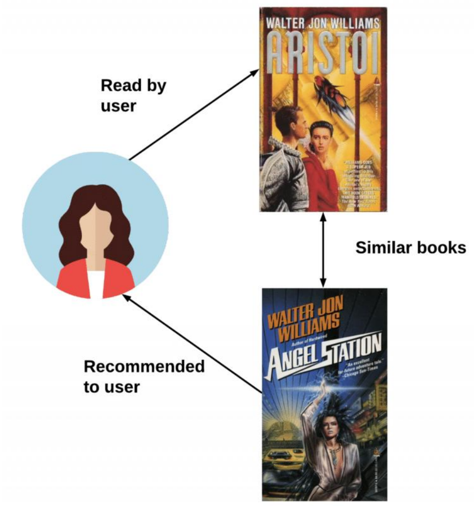
Recommendation System ×
Collaborative Filtering ×
Content Based Filtering ×
Data Science ×
Machine Learning ×

Published Days

Louis Wang in DataDrivenInvestor · Mar 22, 2021

Machine Learning System Design: Recommendation System for Restaurants

Algorithm



Title	Genre	Format	Page Count	Accessibility Feature	Year Published
The Alchemist	Fiction	Paperback	208	Audiobook, Large Print	1988
To Kill a Mockingbird	Fiction	Hardcover	324	Audiobook, Braille	1960
The Catcher in the Rye	Fiction	Paperback	277	Audiobook	1951
1984	Dystopian	Ebook	328	Audiobook, Large Print	1949
Moby-Dick	Adventure	Hardcover	635	Large Print, Braille	1851
The Great Gatsby	Fiction	Paperback	180	Audiobook, Large Print	1925
The Hobbit	Fantasy	Hardcover	310	Audiobook	1937
A Brief History of Time	Science	Ebook	256	Audiobook, Large Print	1988
Sapiens: A Brief History of Humankind	Non-fiction	Paperback	464	Audiobook, Braille	2011

Develop a similarity measure

Use a KNN approach

Pros/Cons

Advantages

- The items metadata are known in advance, so we can also apply it to Cold-Start scenarios (user is new, item is new).
- There is also transparency, since the user sees how the new items relate to what was seen before.

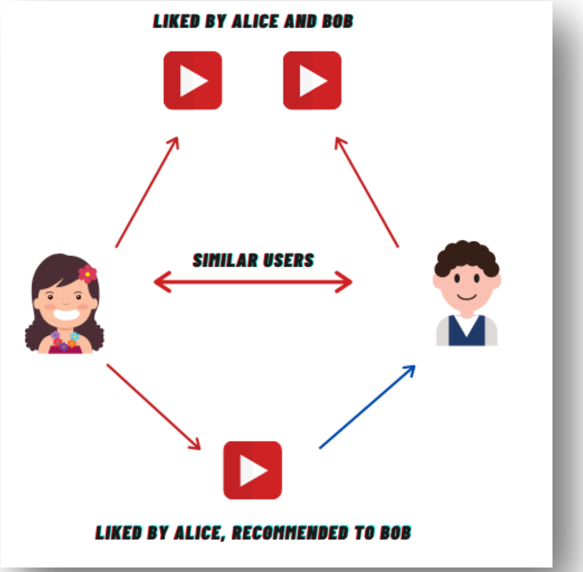
Disadvantages

- We need to know metadata information for each item.
- There is also a risk of a filter-bubble, since content filtering only recommends content similar to the user’s past preferences. If a user reads a book about a political ideology and books related to that ideology are recommended to them, they will be in the “bubble” of their previous interests

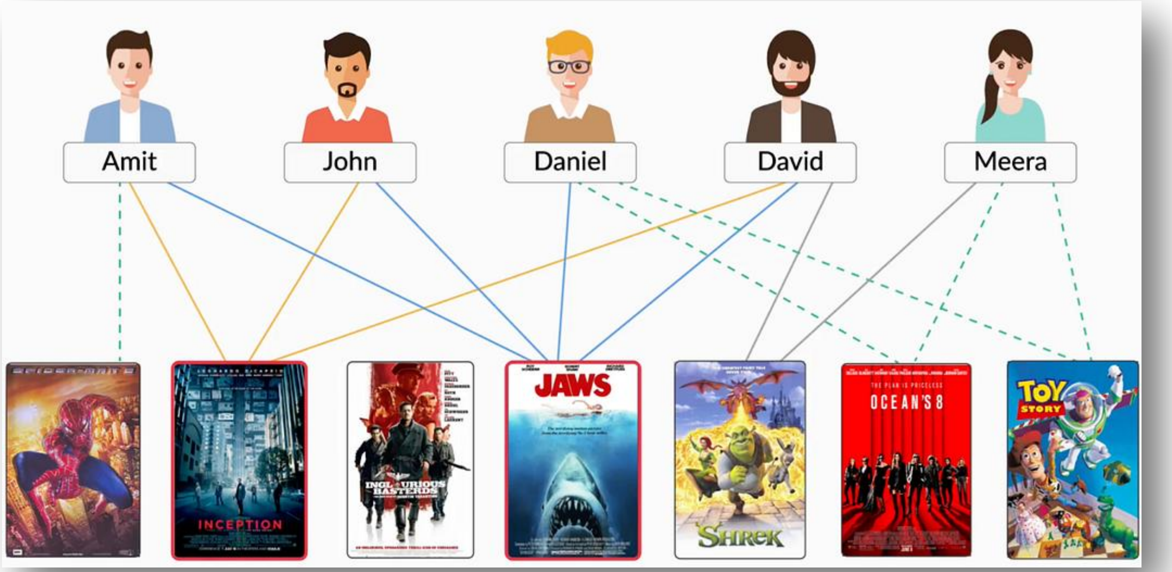
Collaborative Filtering

Utility Matrix

<u>**Collaborative filtering methods**</u> do not use item metadata, but try instead to **leverage the feedbacks or activity history of all users** in order to predict the rating of a user on a given item by inferring interdependencies between users and items from the observed activities.

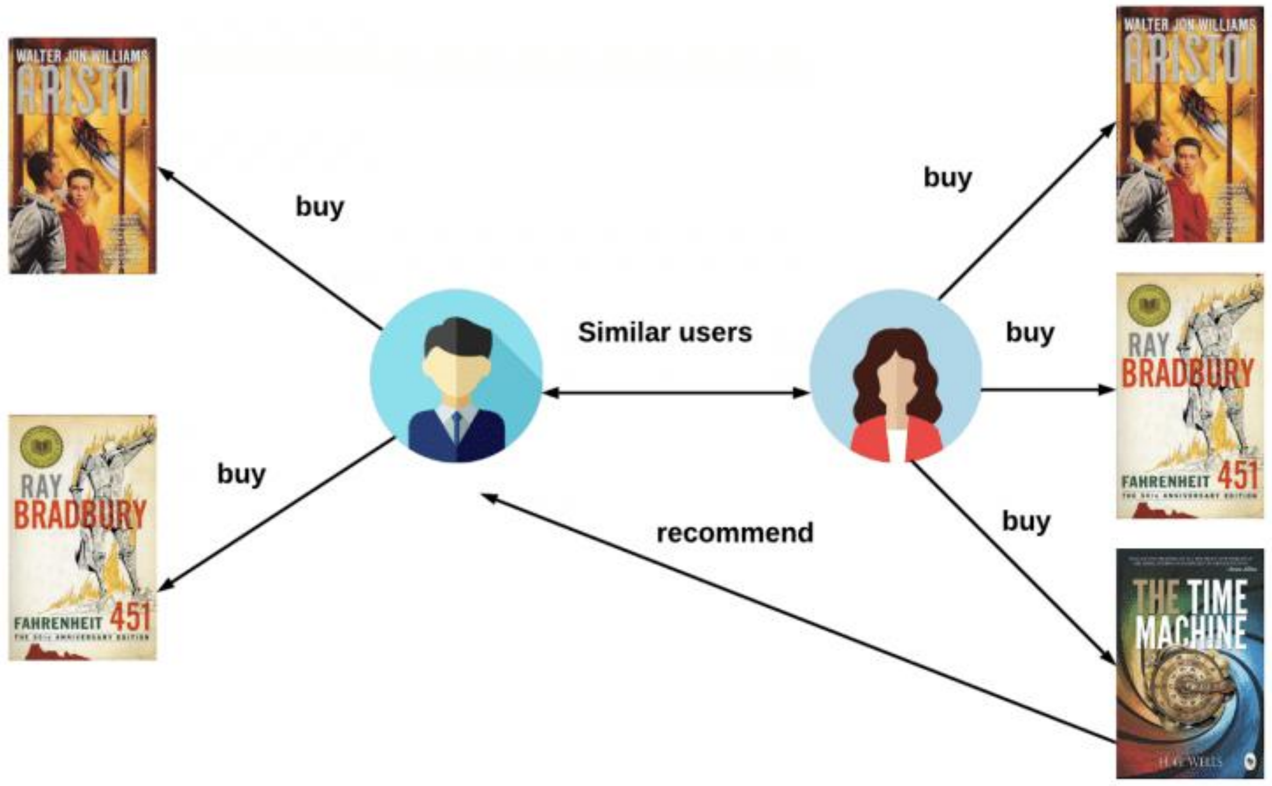


User-based



Item-based

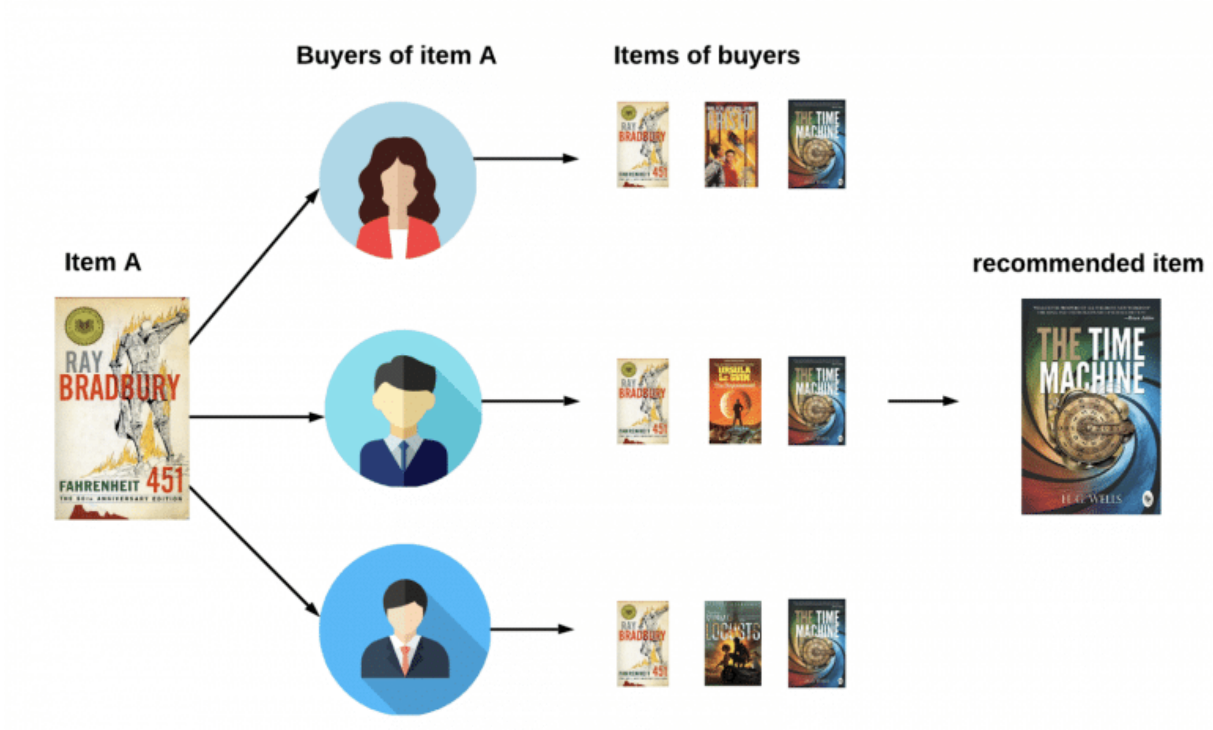
User based



With the user-based approach, recommendations to the target user are made by identifying other users who have shown similar behavior or preferences.

This could be "**users who are similar to you also liked…**" type of recommendations.

Item-based



In item-based collaborative filtering, recommendations are made by identifying items that are similar to the ones the target user has already interacted with.

This can include "**users who liked this item also liked…**" type of recommendations.

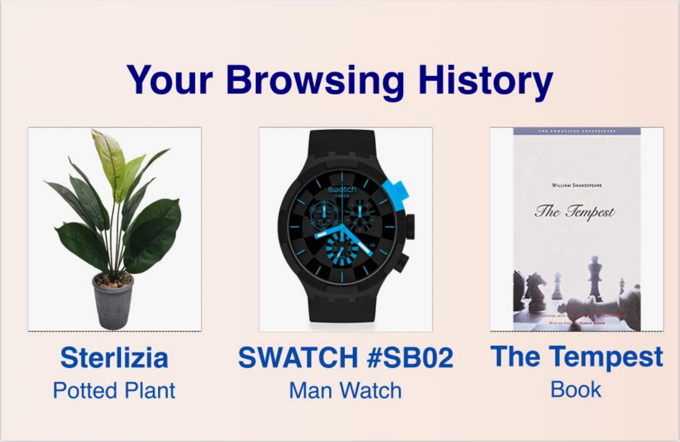
Implicit/Explicit Feedback

Explicit Feedback

- Thumbs up / down
- Star ratings from 1 to 5
- Written reviews

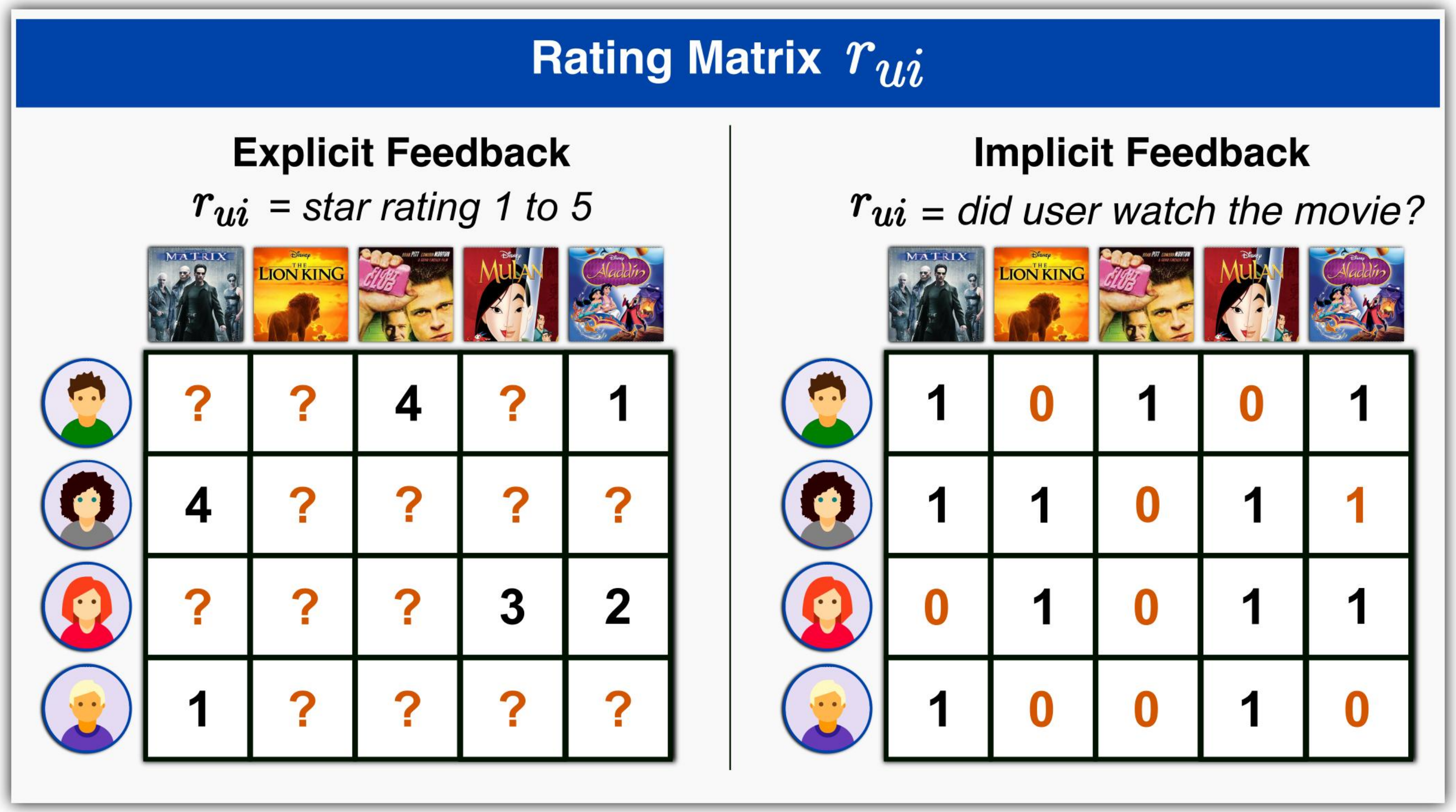
Implicit Feedback

- Played songs / videos
- Purchased items
- Browsing history



Use proxies to determine what you like

Utility Matrix



Utility Matrix

Once we have collected explicit or implicit feedbacks, we can create the **user-item rating matrix r_{ui}.**

For explicit feedback, each entry in r_{ui} is a numerical value—e.g. r_{ui} = "stars given by u to movie i"—or "?" if user u did not rate item i.

For implicit feedback, the values in r_{ui} are a boolean values representing presence or lack of interaction—e.g. r_{ui} = "did user u watch movie i?".

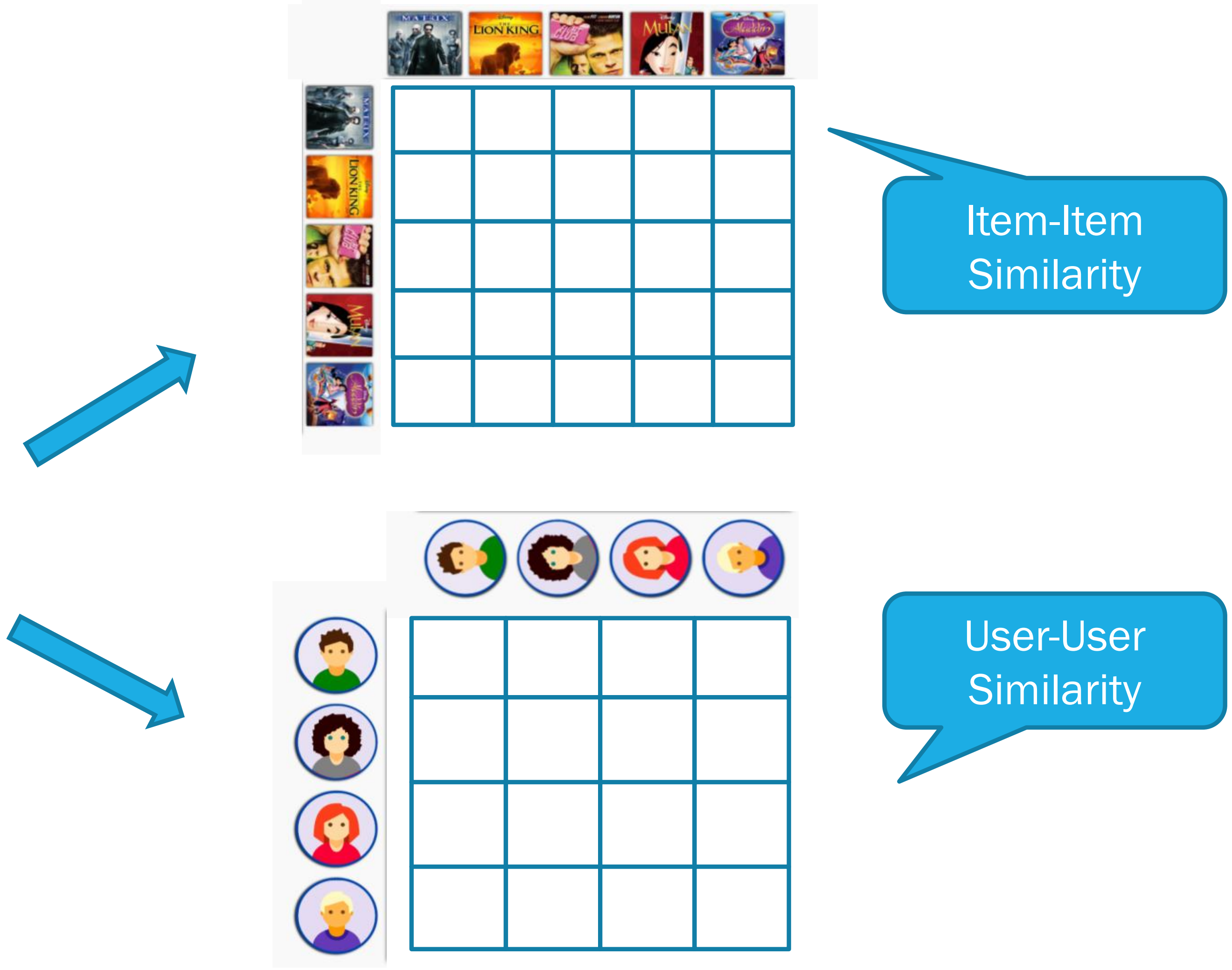
Notice that the matrix r_{ui} is **very sparse**, as users interact with few items among all available contents, and they review even fewer items!

Collaborative Filtering

Memory-based

Utility Matrix r_{ui}

?	?	4	?	1
4	?	?	?	?
?	?	?	3	2
1	?	?	?	?

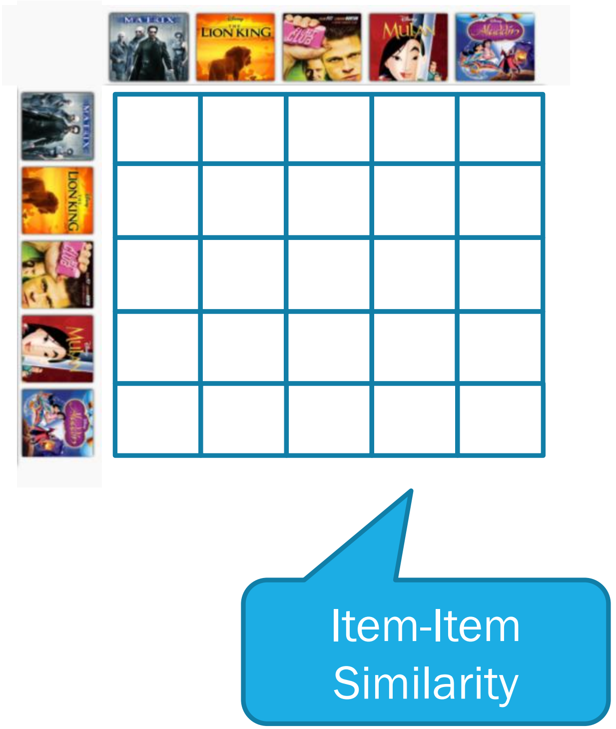


How to recommend to Robert a new book similar to Book X ?

1. Look up similar items: In the item-item matrix, we find N items that are most similar to Book X.

2. Candidate filtering: We check if Robert has already bought any of these books and eliminate them from the list.

3. Candidate scoring: We can score these top N similar books by popularity or by rating from other users, or other criteria.



User	Book X	Item B	Item C	Item D	Item E
U1	5	4		4	5
U2	4	5	4		4
U3	2	3		3	
U4		2	3	2	3
Robert	5		4	5	
U6		4	5	4	
U7	1	2			1
U8		1	2		2
U9	4		4	5	4
U10	3	3		3	

user ratings used for similarity

Can only measure similarity on known ratings... if not enough overlap, than not reliable

Cosine

$$\mathrm{sim}(u,v) = \frac{\sum_{i \in I_{uv}} r_{u,i} \cdot r_{v,i}}{\sqrt{\sum_{i \in I_{uv}} r_{u,i}^2} \cdot \sqrt{\sum_{i \in I_{uv}} r_{v,i}^2}}$$

Robert read A, B, D

Geet : ~~A~~, ~~D~~, E, F ranked 7, 4, 4, 3

Bo: ~~B~~, C, E ranked 4, 5, 5

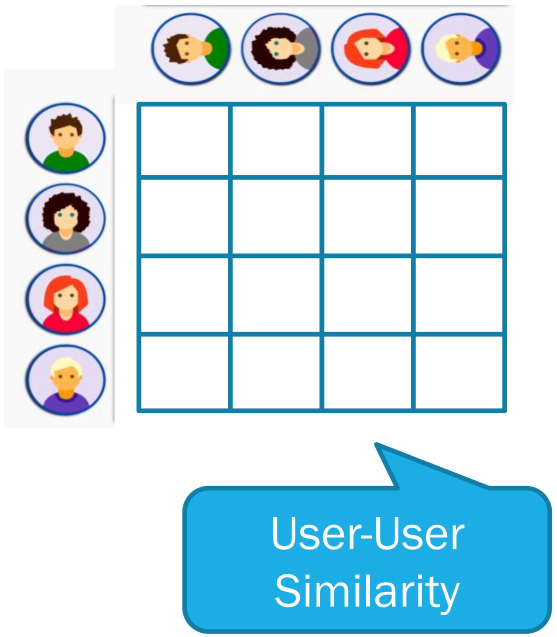
How to recommend to Robert a new book based on people with the same taste as him?

1. Look up similar users: In the user-user matrix, we find N users that are most similar to Robert.

2. Candidate generation: We gather all the books that the N users have read and the ratings they gave them. Let's call that set B.

3. Candidate filtering: We eliminate from set B the books that Robert already read (or bought).

4. Candidate scoring: We average the N users ratings of books in set B and rank them from the ones they liked the most, to the ones they liked the least.



User	Book X	Item B	Item C	Item D	Item E
U1	5	4		4	5
U2	4	5	4		4
U3	2	3		3	
U4		2	3	2	3
Robert	5		4	5	
U6		4	5	4	
U7	1	2			1
U8		1	2		2
U9	4		4	5	4
U10	3	3		3	

Cosine

$$\text{sim}(u,v) = \frac{\sum_{i \in I_{uv}} r_{u,i} \cdot r_{v,i}}{\sqrt{\sum_{i \in I_{uv}} r_{u,i}^2} \cdot \sqrt{\sum_{i \in I_{uv}} r_{v,i}^2}}$$

$$\text{sim}(u,v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2} \cdot \sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}}$$

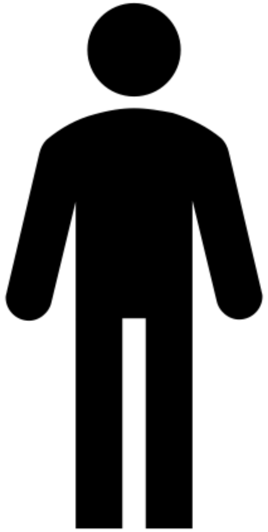
Similarity on ratings

Can only measure similarity on known ratings… if not enough overlap, than not reliable

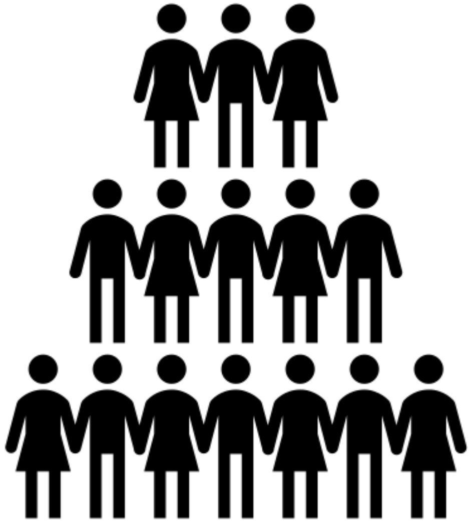
Corrects for skews (users rating everything low or high)

Find a group of similar users based on user-item interaction matrix

Derive the set of items from the similar users



User#1



Group of similar users to user#1

Item	Average Rating	Rank
A	9.6	1
G	8.5	2
B	7	3
Q	6.1	4
C	5.8	5

Set of Items that user#1 have never interacted with before

Collaborative Filtering

Model-based

Model building

Can we LEARN a model that would generate the ratings that are missing?

					....	...		
Amit	?	?	3	?			4	?
Meera	1	5	?	?			?	4
Moran	1	?	4	5			2	?
John	?	2	3	4			4	?
.....	....	....	....	....	....	....	....	....

We can learn a Matrix Factorization that would allow such predictions

$$\hat{r}_{ui} = p_u^T q_i$$

User Factors

P	C1	C2	C3
Amit	0.91	1.10	0.51
Meera	1.01	0.12	0.82
Moran	0.75	-0.06	1.13
John	0.35	0.76	0.16

X

Item Factors

Q						
C1	0.15	1.13	0.91	1.23	0.65	-1.15
C2	0.01	-0.14	1.17	1.52	-0.01	0.01
C3	0.33	0.21	-1.2	1.05	0.89	0.82

$\cong$

Rating Matrix

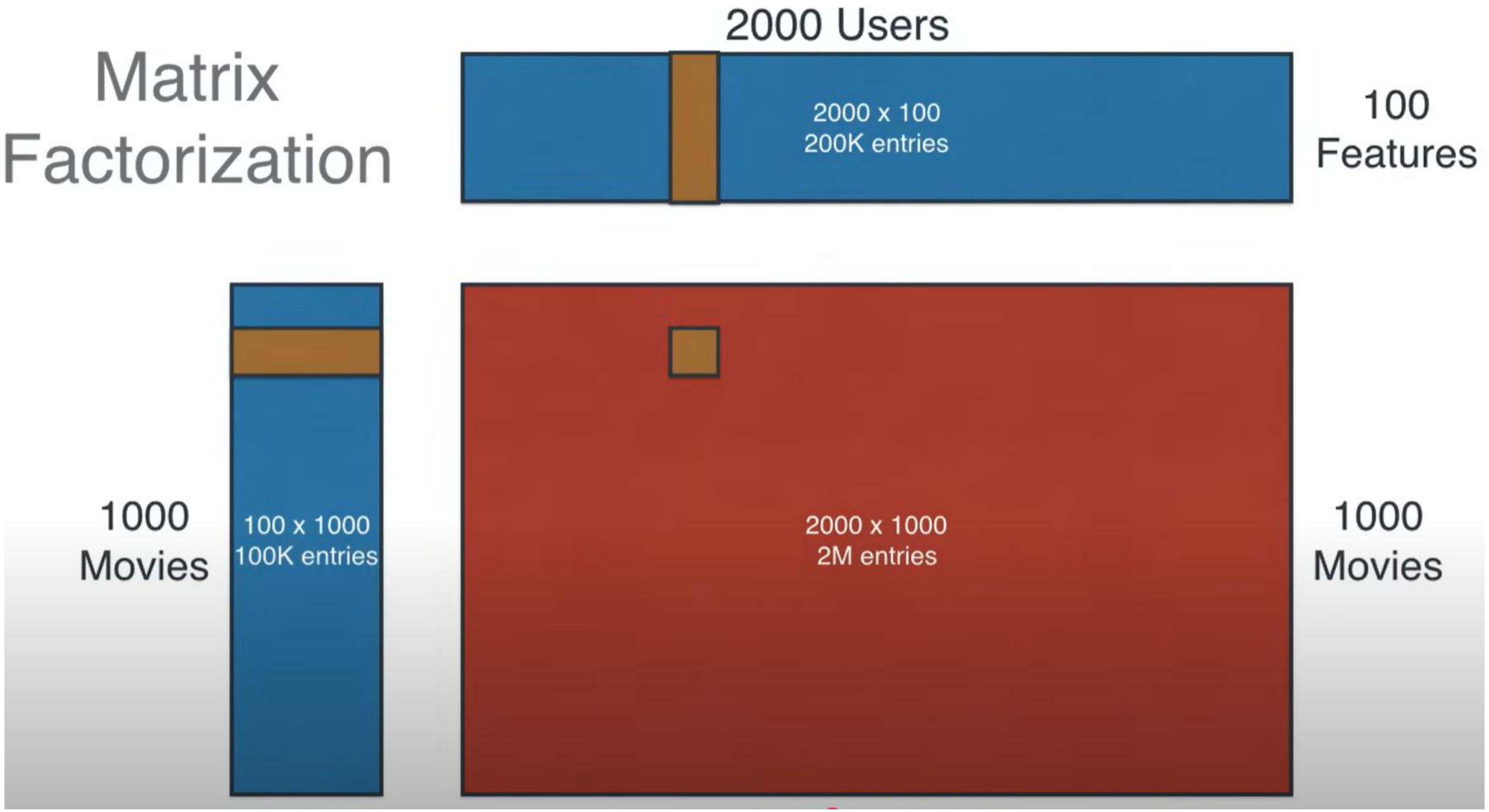
R						
Amit	?	?	5	?	4	?
Meera	1	5	?	?	?	4
Moran	1	?	4	5	2	?
John	?	2	3	4	4	?

Matrix Factorization

Matrix factorization breaks down the original matrix into two parts: one for users and one for items. These matrices capture the essential low-dimensional features of the original data.

By focusing on the most significant factors (K) that drive user behaviour, we obtain a simplified, yet powerful representation.

In matrix factorization, the original matrix is created by multiplying two matrices together. The more ratings in the original matrix, the better the quality and accuracy of the simplified matrices.


Matrix
Factorization
2000 Users
2000 x 100
200K entries
100
Features
1000
Movies
100 x 1000
100K entries
2000 x 1000
2M entries
1000
Movies

Matrix Factorization is about finding these 2 matrices

$$\hat{r}_{ui} = p_u^T q_i$$

Latent dimension = 3

User Factors

P	C1	C2	C3
Amit	0.91	1.10	0.51
Meera	1.01	0.12	0.82
Moran	0.75	-0.06	1.13
John	0.35	0.76	0.16

X

Item Factors

Q						
C1	0.15	1.13	0.91	1.23	0.65	-1.15
C2	0.01	-0.14	1.17	1.52	-0.01	0.01
C3	0.33	0.21	-1.2	1.05	0.89	0.82

$\cong$

Rating Matrix

R						
Amit	?	?	5	?	4	?
Meera	1	5	?	?	?	4
Moran	1	?	4	5	2	?
John	?	2	3	4	4	?

Choose a number (here 3) of latent dimensions

	M1	M2	M3	M4	M5
F1	1.2	3.1	0.3	2.5	0.2
F2	2.4	1.5	4.4	0.4	1.1

Random initialization

	F1	F2
A	0.2	0.5
B	0.3	0.4
C	0.7	0.8
D	0.4	0.5

	M1	M2	M3	M4	M5

Target matrix

	M1	M2	M3	M4	M5
A	3		1		1
B	1		4	1	
C	3	1		3	1
D		3		4	4

Training data

Start random, adjust

	M1	M2	M3	M4	M5
F1	1.2	3.1	0.3	2.5	0.2
F2	2.4	1.5	4.4	0.4	1.1

	F1	F2
A	0.2	0.5
B	0.3	0.4
C	0.7	0.8
D	0.4	0.5

	M1	M2	M3	M4	M5
	1.44	1.37	2.26	0.7	0.59
	1.32	1.53	1.85	0.91	0.5
	2.76	3.37	3.73	2.07	1.02
	1.68	1.99	2.32	1.2	0.63

Compare the 1.44 with 3

$$1.2 \times 0.2 + 2.4 \times 0.5 = 1.44$$

	M1	M2	M3	M4	M5
A	3		1		1
B	1		4	1	
C	3	1		3	1
D		3		4	4

Approximation of Utility matrix X

$$\mathbf{R} \approx \mathbf{P} \times \mathbf{Q}^T = \hat{\mathbf{R}}$$

Approximation

We have each element of r-hat.

$$\hat{r}_{ij} = p_i^T q_j = \sum_{k=1}^{k} p_{ik} q_{kj}$$

Target

Error – Calculated ONLY on the known ratings

We can calculate the error (L2) on each element

$$e_{ij}^2 = (r_{ij} - \hat{r}_{ij})^2 = (r_{ij} - \sum_{k=1}^{K} p_{ik} q_{kj})^2$$

Loss

From this error L2:

$$e_{ij}^2 = (r_{ij} - \hat{r}_{ij})^2 = (r_{ij} - \sum_{k=1}^{K} p_{ik}q_{kj})^2$$

We can calculate the derivatives:

$$\frac{\partial}{\partial p_{ik}} e_{ij}^2 = -2(r_{ij} - \hat{r}_{ij})(q_{kj}) = -2e_{ij}q_{kj}$$
$$\frac{\partial}{\partial q_{ik}} e_{ij}^2 = -2(r_{ij} - \hat{r}_{ij})(p_{ik}) = -2e_{ij}p_{ik}$$

We can update the values using the derivatives and a learning rate:

$$p'_{ik} = p_{ik} - \alpha \frac{\partial}{\partial p_{ik}} e_{ij}^2 = p_{ik} + 2\alpha e_{ij}q_{kj}$$
$$q'_{kj} = q_{kj} - \alpha \frac{\partial}{\partial q_{kj}} e_{ij}^2 = q_{kj} + 2\alpha e_{ij}p_{ik}$$

Gradient Descent
on loss

Update p and q
(alternate)

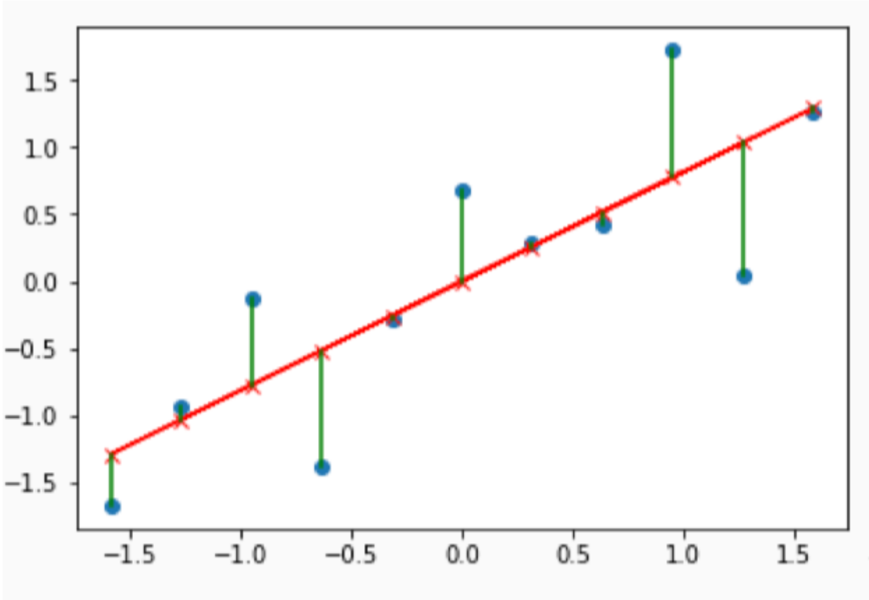
User learning rate

Remember Gradient Descent to estimate the parameters in linear regression ?

Calculate the ***Residual Sum of Squares*** *(RSS). Residuals, or error terms*, are the difference between each ***actual output*** and the ***predicted output***.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^i\right) - y^i\right)^2$$

We want to learn the set of parameters that will minimize that function



Gradient Descent

SAME iterative approach, but for learning the parameters of the factorization

Initialize parameters at random

Repeat:

For each example in training set:

a. Predict $\hat{y}$ (forward pass) $\boxed{\hat{r}_{ij}}$

b. For each parameter b_i $\boxed{\begin{matrix} p_{ik} \\ q_{kj} \end{matrix}}$

   a. Calculate derivative of error $\boxed{\frac{dE}{db_i}}$

   b. Update parameter

$$b_i^{(t)} = b_i^{(t-1)} - \alpha \frac{dE}{db_i}$$

Until "done"

Iterative Process

STOCHASTIC GRADIENT DESCENT

Start random, calculate error

Items factors

q						
C1	0.15	1.13	0.91	1.23	0.65	-1.15
C2	0.01	-0.14	1.17	1.52	-0.01	0.01
C3	0.33	0.21	-1.2	1.05	0.89	0.82

User Factors

p	C1	C2	C3
Amit	0.91	1.10	0.51
Meera	1.01	0.12	0.82
Moran	0.75	-0.06	1.13
John	0.35	0.76	0.16

Rating Matrix

r						
Amit	?	?	5	?	4	?
Meera	1	5	?	?	?	4
Moran	1	?	4	5	2	?
John	?	2	3	4	4	?

Loss function = $Min_{p*,q*} \sum_{\text{known } r_{ui}} (r_{ui} - p_u^T q_i)^2$

$$p_u^T q_i = \begin{pmatrix} 0.91 \\ 1.10 \\ 0.51 \end{pmatrix} X \,(0.91\ 1.17\ -1.2) = 0.91\ X\ 0.91 + 1.10\ X\ 1.17 + 0.51\ X\ -1.2 = 1.5$$

$$(r_{ui} - p^T{}_u q_i)^2 = (\boxed{5} - \boxed{1.5})^2 = 12.25$$

$$e_{ij}^2 = (r_{ij} - \hat{r}_{ij})^2 = (r_{ij} - \sum_{k=1}^{K} p_{ik} q_{kj})^2$$

Error function could include normalization terms

$$\hat{r}_{ui} = q_i^T p_u$$

$$L = \sum_{(u,i) \in K} \left[(r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|p_u\|_2^2 + \|q_i\|_2^2 \right) \right]$$

MATRIX FACTORIZATION

User Factors

P	C1	C2	C3
Amit	0.91	1.10	0.51
Meera	1.01	0.12	0.82
Moran	0.75	-0.06	1.13
John	0.35	0.76	0.16

X

Item Factors

Q						
C1	0.15	1.13	0.91	1.23	0.65	-1.15
C2	0.01	-0.14	1.17	1.52	-0.01	0.01
C3	0.33	0.21	-1.2	1.05	0.89	0.82

$\cong$

Rating Matrix

R						
Amit	?	?	5	?	4	?
Meera	1	5	?	?	?	4
Moran	1	?	4	5	2	?
John	?	2	3	4	4	?

Pros/Cons

Advantages

- **Effective personalization:** Collaborative filtering is highly effective in providing personalized recommendations to users. It takes into account the behavior and preferences of similar users to suggest items that a particular user is likely to enjoy.
- **No need for item attributes:** Useful for items (e.g. clothing) for which metadata is not as standardized as books or movies.
- **Serendipitous discoveries:** Collaborative filtering can introduce users to items they might not have discovered otherwise.

Disadvantages

- **The “cold-start” problem**: New users will not have rated any items yet and will therefore not be similar to anyone.
- **Sensitivity to sparse data**: Collaborative filtering may struggle to find useful patterns or similarities between users and items when data is very sparse.
- **Potential for popularity bias**: Collaborative filtering tends to recommend popular items more frequently. This can lead to a “rich get richer” phenomenon, where already popular items receive even more attention, while niche or less-known items are overlooked.

Evaluation

	M1	M2	M3	M4	M5
F1	3	1	1	3	1
F2	1	2	4	1	3

What are the recommendations?

	F1	F2
A	1	0
B	0	1
C	1	0
D	1	1

	M1	M2	M3	M4	M5
A	3	1	1	3	1
B	1	2	4	1	3
C	3	1	1	3	1
D	4	3	5	4	4

Ranked results

1. M4
2. M2

1. M3
2. M1

RANKING AND RECOMMENDATION METRICS GUIDE

10 metrics to evaluate recommender and ranking systems

Last updated: February 14, 2025

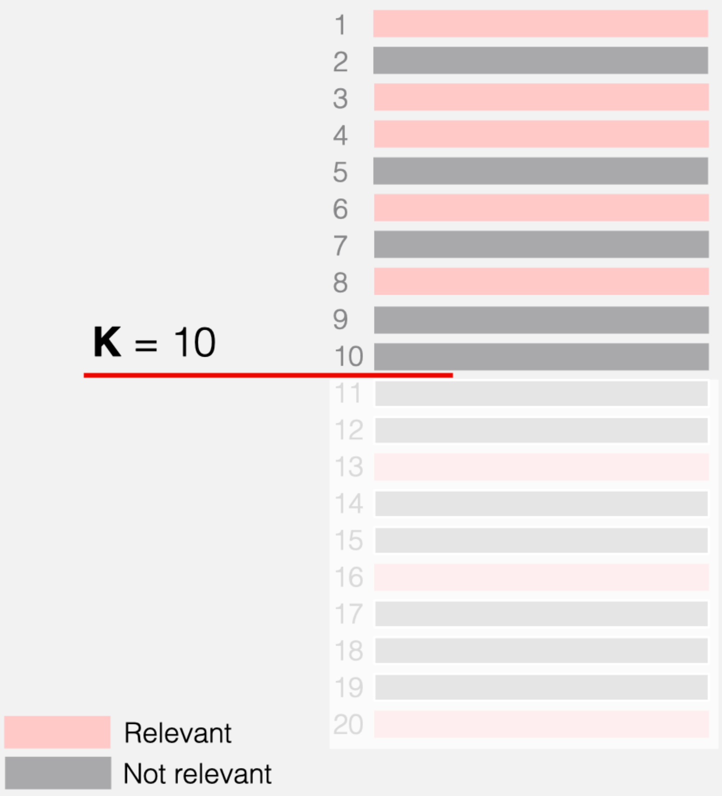
Source

Assumes a gold standard

1. **Predictive metrics.** They reflect the “correctness” of recommendations and show how well the system finds relevant items.
2. **Ranking metrics.** They reflect the ranking quality: how well the system can sort the items from more relevant to less relevant.
3. **Behavioral metrics.** These metrics reflect specific properties of the system, such as how diverse or novel the recommendations are.

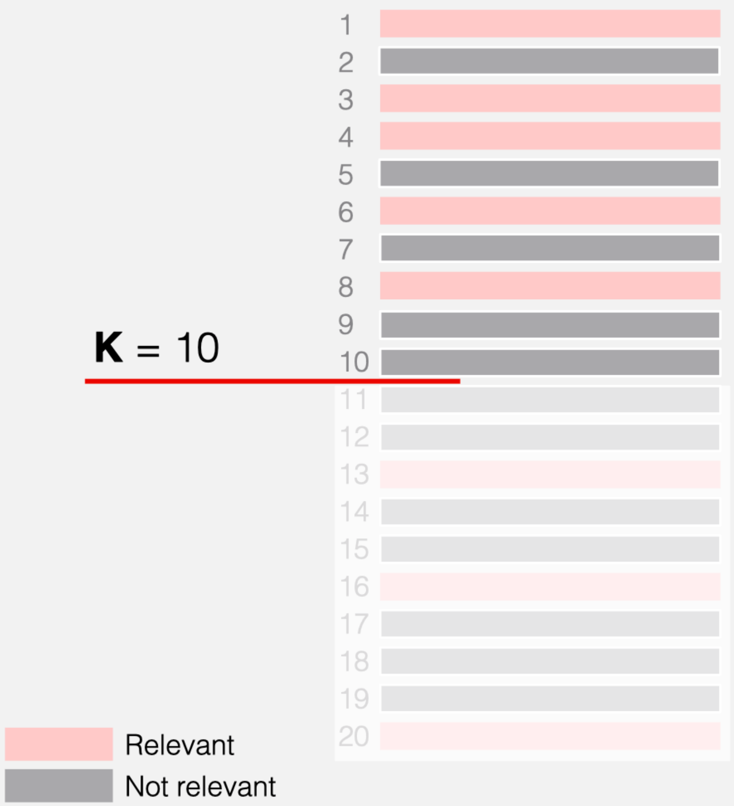
Predictive Quality

Assumes a transformation from rating to relevance



$$\text{Precision@ K} = \frac{\text{\# of relevant items at K}}{\text{K}}$$

$$\text{Precision@10} = \frac{5}{10} = 0.5$$

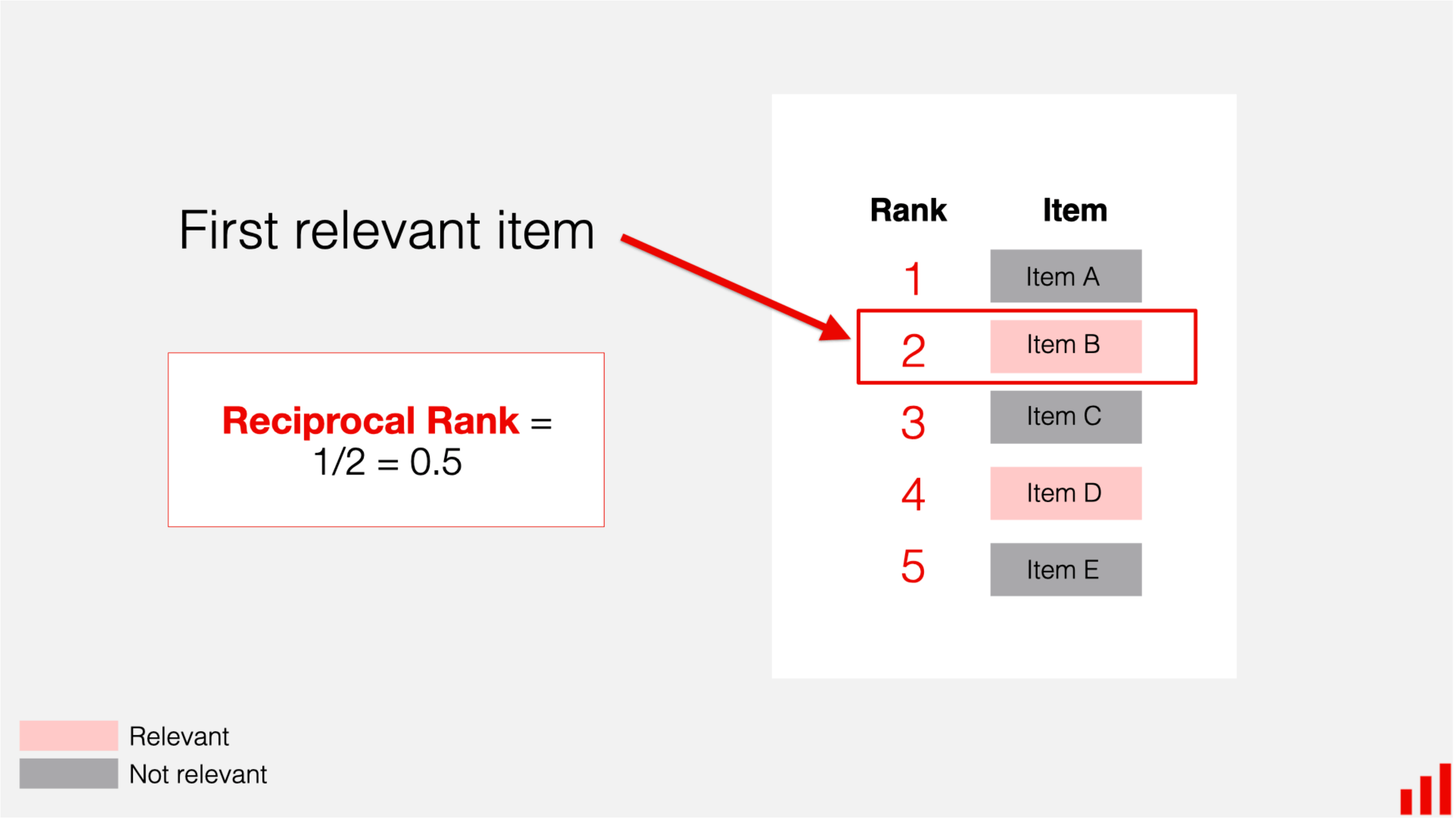


$$\text{Recall@ K} = \frac{\text{\# of relevant items at K}}{\text{\# of relevant items}}$$

$$\text{Recall@10} = \frac{5}{8} = 0.625$$

Ranking Quality

MRR (Mean Reciprocal Rank) shows how soon you can find the first relevant item.
To calculate MRR, you take the reciprocal of the rank of the first relevant item and average this value across all queries or users.



Behavioral Quality

Recommendation novelty

Not the « usual » popular items…

How novel are the recommendations?

- Compute novelty per item based on popularity in training
- Average by user
- Average across users

Recommendation diversity

How varied are the recommendations for each user?

- Compute intra-list diversity
- Average across users

Among what is suggested, how different are the items

Recommendation serendipity

Surprisingly relevant!

Can recommendations positively surprise?

- Measure the distance between relevant recommendations and previous user history
- Average across users

RECOMMENDER SYSTEMS

- Introduction
- Content-Based Filtering
- Collaborative Filtering
  - Utility matrix
  - Memory-based
  - Model-based
- Evaluation